Forensics: Analysis of a PCAP file

Cyber Security and Forensic Computing BSc (Hons)

Ahnaf Ahmed

Table of Contents

Assessing the PCAP File.....3

HTTP Analysis & Blind SQL Injection.....6

FTP Brute Force Attack.....7

Conclusion.....9

References.....10

Assessing the PCAP file

`

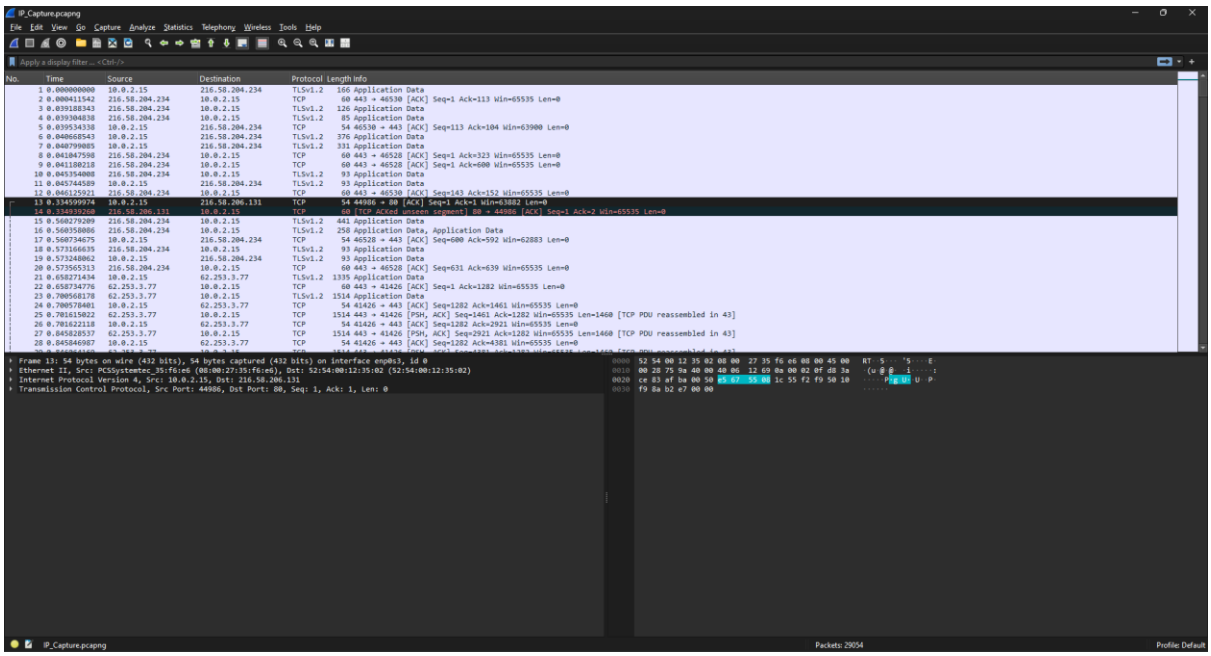


Figure 1: Packets displayed from PCAP file

Upon installing the Wireshark application and opening the PCAP file, we are provided with a total of 29054 packets to assess and determine malicious activity. From here, we can approach many paths in order to determine malicious activity.

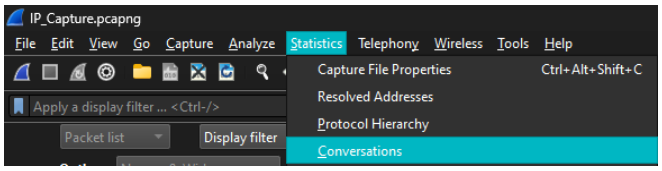


Figure 2: Statistics Dropdown Menu

In this instance, we can start by accessing the conversations page, as shown in figure 2 and sorting the packet size from high to low, as shown in figure 3. The reason why we have started with arranging packet size from high to low is so we can see where the most data transfer has occurred. Larger data transfer between to endpoints is a common indicator of suspicious activity (Shakeel, 2025).

Ethernet · 3 | IPv4 · 20 | IPv6 · 1 | TCP · 3153 | UDP · 2053

Address A	Address B	Packets ˅	Bytes	Stream ID	Packets A → B	Bytes A → B	Packets B → A	Bytes B → A	Rel Start	Duration	Bits/s A → B	Bits/s B → A
10.0.2.15	62.253.3.77	14,613	16 MB	2	6,043	355 kB	8,570	16 MB	0.658271	497.3403	5709 bits/s	254 kbps
10.0.2.15	192.168.56.101	9,543	699 kB	9	5,794	342 kB	3,749	357 kB	19.693213	466.3075	5864 bits/s	6129 bits/s
10.0.2.15	172.217.169.14	1,293	704 kB	5	483	600 kB	810	104 kB	2.748310	494.7858	9701 bits/s	1676 bits/s
10.0.2.15	216.58.206.138	1,257	207 kB	4	514	86 kB	743	121 kB	2.493828	485.4290	1418 bits/s	1996 bits/s
10.0.2.15	216.58.206.110	809	433 kB	6	272	375 kB	537	59 kB	3.499945	494.4630	6059 bits/s	951 bits/s
10.0.2.15	216.58.198.170	483	75 kB	19	206	34 kB	277	40 kB	364.999185	130.7049	2106 bits/s	2460 bits/s
10.0.2.15	172.217.169.78	313	77 kB	3	121	60 kB	192	17 kB	1.486547	489.9395	974 bits/s	283 bits/s
10.0.2.15	216.58.204.234	223	30 kB	0	106	14 kB	117	16 kB	0.000000	460.3467	241 bits/s	275 bits/s
10.0.2.15	216.58.204.228	112	18 kB	13	57	10 kB	55	8 kB	48.277989	433.0885	189 bits/s	145 bits/s
10.0.2.15	194.168.4.100	106	12 kB	7	53	5 kB	53	7 kB	6.619590	487.7456	80 bits/s	111 bits/s
10.0.2.15	216.58.212.195	86	7 kB	8	43	3 kB	43	4 kB	10.034676	483.6408	54 bits/s	63 bits/s
172.253.120.189	10.0.2.15	79	13 kB	10	40	9 kB	39	4 kB	22.081250	463.7039	160 bits/s	60 bits/s
10.0.2.15	216.58.210.238	33	4 kB	15	17	2 kB	16	2 kB	82.496910	170.1431	93 bits/s	86 bits/s
10.0.2.15	172.217.169.54	30	8 kB	11	14	1 kB	16	7 kB	23.421455	170.2086	48 bits/s	309 bits/s
10.0.2.15	216.58.206.131	26	1 kB	1	13	702 bytes	13	780 bytes	0.334600	105.2894	53 bits/s	59 bits/s
10.0.2.15	216.58.210.33	16	1 kB	12	8	573 bytes	8	546 bytes	43.276751	112.3477	40 bits/s	38 bits/s
10.0.2.15	216.58.210.230	16	1 kB	14	8	573 bytes	8	546 bytes	48.278122	112.3614	40 bits/s	38 bits/s
10.0.2.15	35.222.85.5	11	873 bytes	16	6	431 bytes	5	442 bytes	200.617028	0.2754	12 kbps	12 kbps
52.13.148.70	10.0.2.15	3	234 bytes	18	2	145 bytes	1	89 bytes	272.715565	0.0005		
10.0.2.15	224.0.0.251	1	160 bytes	17	1	160 bytes	0	0 bytes	235.017697	0.0000		

Figure 3: Conversations page

Figure 3 displays a suspicious amount of data being transferred from host IP (10.0.2.15) to external IP (62.253.3.77). The packets transferred between both IP addresses total up to 14,613 packets. Figure 3 also shows that most packets were transferred from the external IP address to the Host IP (with a total of 16MB). We can identify this external IP address by using the command 'nslookup', which leads us to Virgin media business IP as shown in figure 4.

```
PS C:\Program Files\Wireshark> nslookup 62.253.3.77
Server:  andc.wireless.resolver.ddi.port.ac.uk
Address:  148.197.3.39

Name:    77.3-253-62.static.virginmediabusiness.co.uk
Address:  62.253.3.77
```

Figure 4: nslookup command

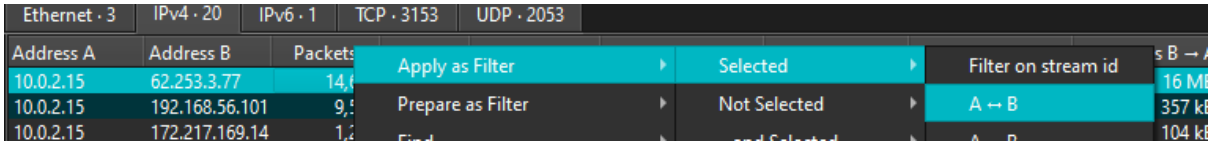


Figure 5: Conversation Filter

To further investigate this, we need to assess the data transferred between the two endpoints. This is achieved by applying a filter, (as shown in Figure 5) so we can only see the transmission between the two endpoints specifically.

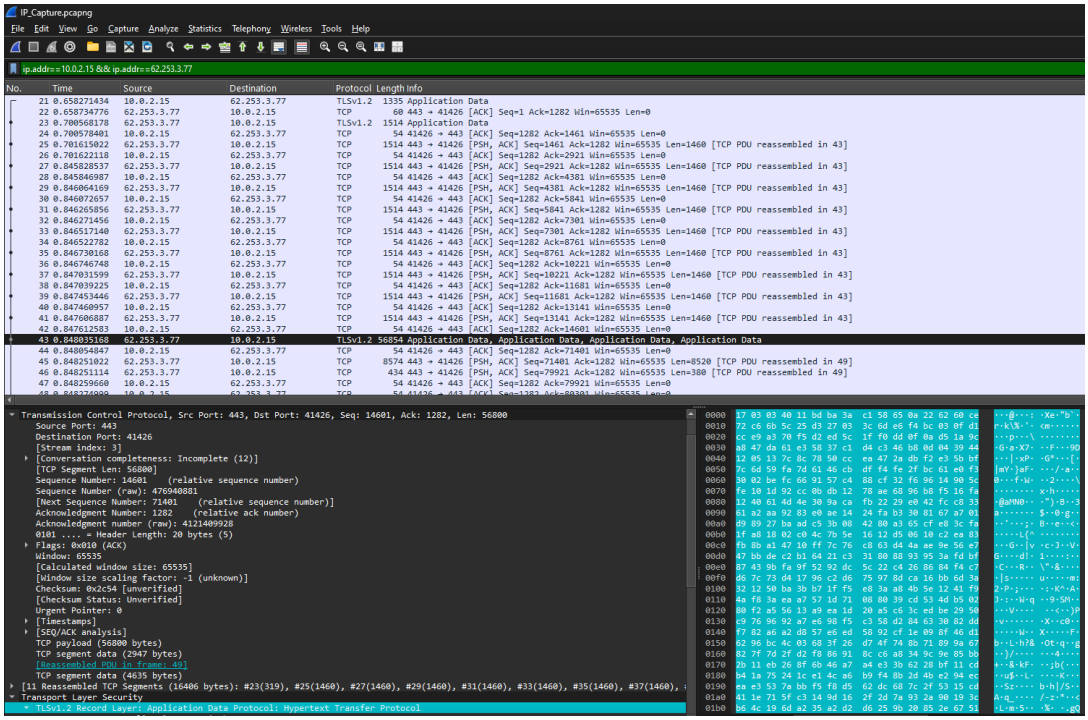


Figure 6: HTTP encryption between Host and External IP address

Figure 6 displays the communication between the host IP (10.0.2.15) and the external (62.253.3.77). From this page, we know that the protocol used to encrypt the data is Hypertext Transfer Protocol, which is displayed underneath the 'Transport Layer Security' tab.

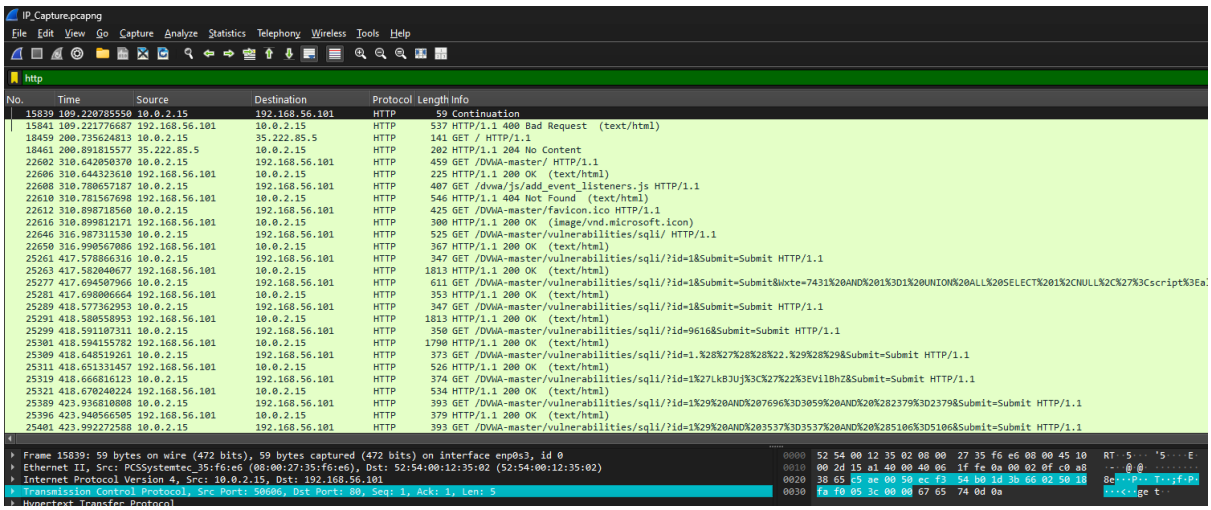


Figure 7: HTTP Packets are Filtered

HTTP Analysis

This is important because HTTP traffic analysis assists in streaming and identifying cyber fighting offers, exploiting vulnerabilities in the server, etc. HTTP itself is a stateless protocol, primarily through the following methods: GET POST 1. GET requests are generally used to fetch resources, like web pages or images, while POST requests are used for sending data, like forms or file uploads. This method is often exploited by attackers attempting to perform SQL injection or cross-site scripting (XSS) attacks. (*How to Analyze HTTP Traffic in Wireshark for Cybersecurity Purposes | LabEx*, n.d.)

As demonstrated in figure 7, a Wireshark filter is applied to show only HTTP requests detected: 68 packets from the same (10.0.2.15) IP. The regularity of this source, as well as continuous GET requests to the /sqli/ directory, suggested that the attacker was testing the server for SQL injection vulnerabilities (Wisnawa, 2025).
Some behaviours seen in the HTTP traffic, such as odd URLs and query strings, provided more evidence of malicious intent. This information allows investigators to build a more comprehensive picture of the attacker's modes of operation and objectives to create countermeasures to combat these actions more effectively. In this way, these breaking down disclosures recognise such signs at optional ruptures and keep following changes in the basic security posture of web applications.

Blind SQL Injection

Blind SQL injection is an attack vector that allows an attacker to induce a user to the target web application and then have him identify something about a database without seeing the output (*Blind SQL Injection | OWASP Foundation*, n.d.).
In this case, we need to examine time-based attacks, which we identify as having the keyword 'SLEEP' in some of the HTTP requests (Figure 7). This approach leverages the SLEEP() function used in a database, which delays its response for a specific amount of time based on whether or not the attack input was executed properly (Invicti, 2024). In order to filter out all the packets that use the 'SLEEP' function and originate from the IP '10.0.2.15', we use the filter: 'http.request.uri contains "SLEEP" && ip.src == 10.0.2.15', as shown in figure 8.

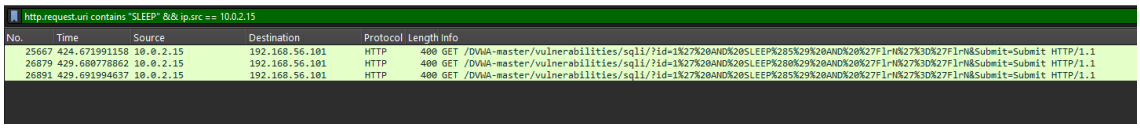


Figure 8: SLEEP and 10.0.2.15 Packet Filter

Three packets were visible after filtration where the attacker targeted the function 'SLEEP' with different parameters. This id=1 parameter showed that the attacker was aiming to gain access to particular records in the database, usually the first record in a table (Siemons, 2024).
The output would need to be URL-decoded in order to comprehend the encoded characters in the query strings, which shows how attackers modify the SQL syntax to run their payload (Aegis, 2021). Figure 9 displays the decoded versions of the HTTP queries.

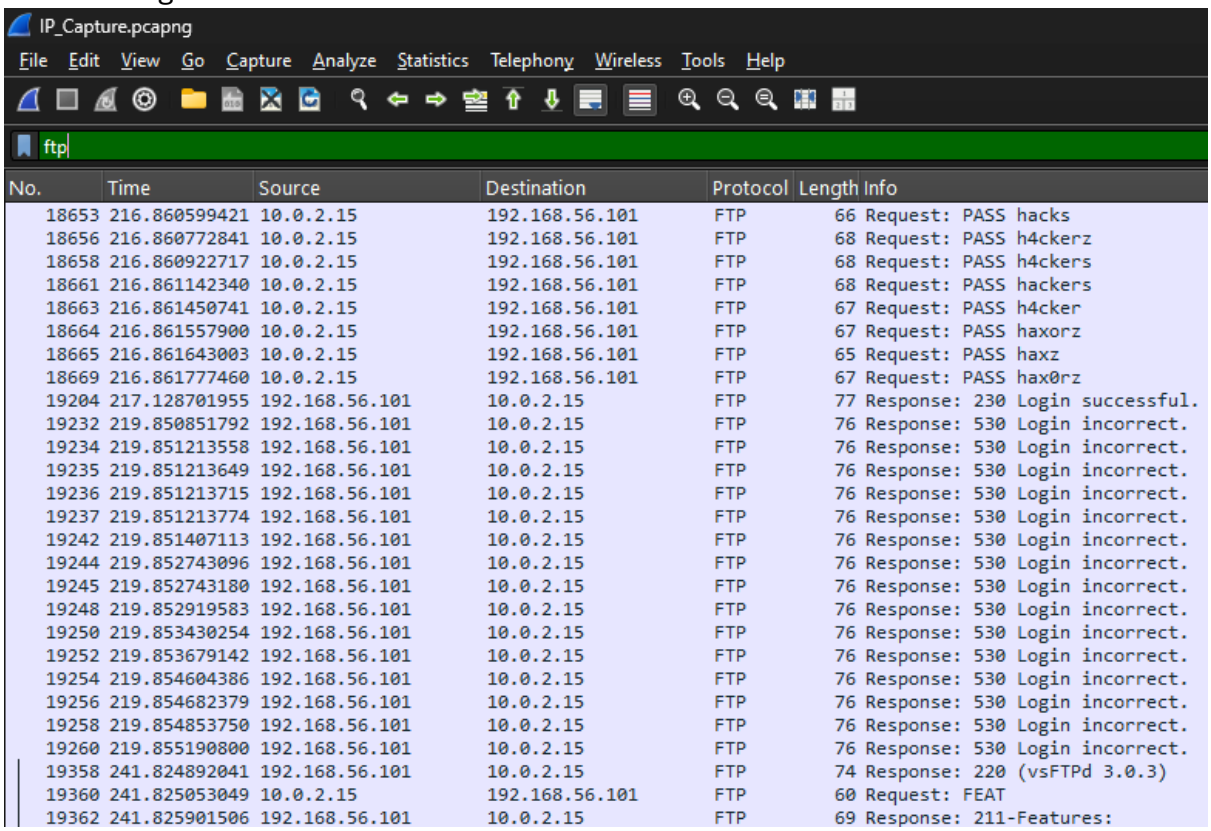
```
/DVWA-master/vulnerabilities/sqli/?id=1' AND SLEEP(5) AND 'FlrN'='FlrN&Submit=Submit HTTP/1.1\r\n

/DVWA-master/vulnerabilities/sqli/?id=1' AND SLEEP(0) AND 'FlrN'='FlrN&Submit=Submit

/DVWA-master/vulnerabilities/sqli/?id=1' AND SLEEP(5) AND 'FlrN'='FlrN&Submit=Submit
```

Figure 9: Decoded HTTP queries

The successful execution of these attacks was confirmed when the server returned HTTP 200 OK in each case, therefore highlighting the server to be a potential target for SQL injection bugs due to potential failure in input validation mechanisms. That highlights the importance of secure coding practices to avoid such attacks and using prepared statements and proper input validation.

FTP Brute Force Attack

FTP (File Transfer Protocol) is a common method used for file transfers, but it is not secure by design since it does not have any encryption method built-in (*FTP Server – Beware of Security Risks*, 2025). We found multiple failed login attempts to the FTP server in the PCAP file that indicate a brute-force attack, where the attacker (originating from IP '10.0.2.15') systematically attempts different username and password combinations (Rmcmurray, 2022). Using the username 'admin', the attacker tried multiple passwords until finally managing to log into the server at 192.168.56.101, as shown in Figure 10.



No.	Time	Source	Destination	Protocol	Length	Info
18653	216.860599421	10.0.2.15	192.168.56.101	FTP	66	Request: PASS hacks
18656	216.860772841	10.0.2.15	192.168.56.101	FTP	68	Request: PASS h4ckerz
18658	216.860922717	10.0.2.15	192.168.56.101	FTP	68	Request: PASS h4ckers
18661	216.861142340	10.0.2.15	192.168.56.101	FTP	68	Request: PASS hackers
18663	216.861450741	10.0.2.15	192.168.56.101	FTP	67	Request: PASS h4cker
18664	216.861557900	10.0.2.15	192.168.56.101	FTP	67	Request: PASS haxorz
18665	216.861643003	10.0.2.15	192.168.56.101	FTP	65	Request: PASS haxz
18669	216.861777460	10.0.2.15	192.168.56.101	FTP	67	Request: PASS hax0rz
19204	217.128701955	192.168.56.101	10.0.2.15	FTP	77	Response: 230 Login successful.
19232	219.850851792	192.168.56.101	10.0.2.15	FTP	76	Response: 530 Login incorrect.
19234	219.851213558	192.168.56.101	10.0.2.15	FTP	76	Response: 530 Login incorrect.
19235	219.851213649	192.168.56.101	10.0.2.15	FTP	76	Response: 530 Login incorrect.
19236	219.851213715	192.168.56.101	10.0.2.15	FTP	76	Response: 530 Login incorrect.
19237	219.851213774	192.168.56.101	10.0.2.15	FTP	76	Response: 530 Login incorrect.
19242	219.851407113	192.168.56.101	10.0.2.15	FTP	76	Response: 530 Login incorrect.
19244	219.852743096	192.168.56.101	10.0.2.15	FTP	76	Response: 530 Login incorrect.
19245	219.852743180	192.168.56.101	10.0.2.15	FTP	76	Response: 530 Login incorrect.
19248	219.852919583	192.168.56.101	10.0.2.15	FTP	76	Response: 530 Login incorrect.
19250	219.853430254	192.168.56.101	10.0.2.15	FTP	76	Response: 530 Login incorrect.
19252	219.853679142	192.168.56.101	10.0.2.15	FTP	76	Response: 530 Login incorrect.
19254	219.854604386	192.168.56.101	10.0.2.15	FTP	76	Response: 530 Login incorrect.
19256	219.854682379	192.168.56.101	10.0.2.15	FTP	76	Response: 530 Login incorrect.
19258	219.854853750	192.168.56.101	10.0.2.15	FTP	76	Response: 530 Login incorrect.
19260	219.855190800	192.168.56.101	10.0.2.15	FTP	76	Response: 530 Login incorrect.
19358	241.824892041	192.168.56.101	10.0.2.15	FTP	74	Response: 220 (vsFTPd 3.0.3)
19360	241.825053049	10.0.2.15	192.168.56.101	FTP	60	Request: FEAT
19362	241.825901506	192.168.56.101	10.0.2.15	FTP	69	Response: 211-Features:

Figure 10: Login Attempts Until Success

This type of password cracking is particularly troubling as it takes advantage of the flaws of FTP, which doesn't provide a way to lock accounts after several failed attempts.

Figures 11 and 12 show upon authentication, the perpetrator browsed the server's directory structure and downloaded system files like index.html and login. php. The index.html is important to see the structure and references of our web program and login. php — this could include sensitive PHP code handling user credentials (V, 2024). If the hacker was able to upload files, then they could change the login php to leave a backdoor so they could come back at any time.

20102	262.389600954	10.0.2.15	192.168.56.101	FTP	62	Request: TYPE I
20104	262.390219899	192.168.56.101	10.0.2.15	FTP	85	Response: 200 Switching to Binary mode.
20106	263.021832821	10.0.2.15	192.168.56.101	FTP	71	Request: SIZE index.html
20108	263.022493595	192.168.56.101	10.0.2.15	FTP	65	Response: 213 10918
20110	263.022601268	10.0.2.15	192.168.56.101	FTP	60	Request: PASV
20112	263.023082540	192.168.56.101	10.0.2.15	FTP	107	Response: 227 Entering Passive Mode (192,168,56,101,183,230)
20117	263.024157741	10.0.2.15	192.168.56.101	FTP	71	Request: RETR index.html
20119	263.024781662	192.168.56.101	10.0.2.15	FTP	125	Response: 150 Opening BINARY mode data connection for index.html (10918 bytes).
20129	263.025328747	192.168.56.101	10.0.2.15	FTP	78	Response: 226 Transfer complete.
20165	275.437118261	10.0.2.15	192.168.56.101	FTP	85	Request: CWD /var/www/html/DVWA-master
20167	275.438041988	192.168.56.101	10.0.2.15	FTP	91	Response: 250 Directory successfully changed.
20169	275.438375190	10.0.2.15	192.168.56.101	FTP	62	Request: TYPE A
20171	275.438774213	192.168.56.101	10.0.2.15	FTP	84	Response: 200 Switching to ASCII mode.
20173	275.438882706	10.0.2.15	192.168.56.101	FTP	60	Request: PASV
20175	275.439414331	192.168.56.101	10.0.2.15	FTP	107	Response: 227 Entering Passive Mode (192,168,56,101,226,196)
20180	275.440355280	10.0.2.15	192.168.56.101	FTP	60	Request: LIST

Figure 11: Index.html Download

20182	275.440905497	192.168.56.101	10.0.2.15	FTP	93	Response: 150 Here comes the directory listing.
20187	275.441576563	192.168.56.101	10.0.2.15	FTP	78	Response: 226 Directory send OK.
20620	287.085879699	10.0.2.15	192.168.56.101	FTP	62	Request: TYPE I
20622	287.086800468	192.168.56.101	10.0.2.15	FTP	85	Response: 200 Switching to Binary mode.
20626	289.934598971	10.0.2.15	192.168.56.101	FTP	70	Request: SIZE login.php
20628	289.935308718	192.168.56.101	10.0.2.15	FTP	64	Response: 213 4163
20630	289.935458749	10.0.2.15	192.168.56.101	FTP	60	Request: PASV
20632	289.936123257	192.168.56.101	10.0.2.15	FTP	107	Response: 227 Entering Passive Mode (192,168,56,101,202,122)
20637	289.937037329	10.0.2.15	192.168.56.101	FTP	70	Request: RETR login.php
20639	289.937588962	192.168.56.101	10.0.2.15	FTP	123	Response: 150 Opening BINARY mode data connection for login.php (4163 bytes).
20643	289.939824784	192.168.56.101	10.0.2.15	FTP	78	Response: 226 Transfer complete.
23929	373.174947240	10.0.2.15	192.168.56.101	FTP	60	Request: QUIT
23931	373.175840091	192.168.56.101	10.0.2.15	FTP	68	Response: 221 Goodbye.

Figure 12:Login.php Download

This highlights that FTP servers should be set with password strength and account lockout policies. For even greater security, SFTP (Secure File Transfer Protocol) should be implemented wherever possible.

Conclusion

Examining the PCAP file and discovering SQL injection, FTP brute force, and successful attacks. The attacker sending the requests was using a local IP address (10.0.2.15) to obfuscate their origin and attempted to extract sensitive data utilizing a variety of exploitation techniques using both time-based SQL injections. The analysis also showed that the attacker was exfiltrating data out to external services, with large data transfers to Google signifying an organized effort to steal and potentially sell sensitive information. The static IP (62.253.3.77) on the Virgin Media Business network suggests that the attacker was operating from a corporate environment and potentially had access to corporate resources. The consistency of communications between the attacker and this IP helps reinforce the idea that the attacker has infrastructure built to conduct cyber operations. Due to the severity of the findings, this means organizations should implement strong security measures, including input validation, account lockout policies, and network monitoring to help mitigate against such attacks. Reporting the findings will help investigators investigate the attacker's activities, while organizations should proactively tighten their cybersecurity practices to minimize future threats.

References:

1. Shakeel, I. (2025, March 24). *Network traffic analysis using Wireshark*. https://levelblue.com/blogs/security-essentials/network-traffic-analysis-using-wireshark?
2. Smith, C. (2024, May 31). How to filter HTTP traffic in Wireshark | NetworkProGuide. *NetworkProGuide*. https://networkproguide.com/wireshark-filter-http-traffic/
3. *How to analyze HTTP traffic in Wireshark for Cybersecurity purposes | LabEx*. (n.d.). LabEx. https://labex.io/tutorials/wireshark-how-to-analyze-http-traffic-in-wireshark-for-cybersecurity-purposes-415495
4. Wisnawa, A. (2025, January 24). PCAP File Analysis with Wireshark: Discovering Scanning, XSS, and SQL Injection Attack Attemps. *Medium*. https://medium.com/%40aristawisnawa/pcap-file-analysis-with-wireshark-discovering-scanning-xss-and-sql-injection-attack-attemps-e136cce4d741
5. Invicti. (2024, May 17). *Blind SQL injection*. https://www.invicti.com/learn/blind-sql-injection/
6. *Blind SQL Injection | OWASP Foundation*. (n.d.). https://owasp.org/www-community/attacks/Blind_SQL_Injection
7. Siemons, F. (2024, March 4). *SQL injection analysis*. Infosec Institute. https://www.infosecinstitute.com/resources/malware-analysis/sql-injection-analysis/
8. URL encode decode - URL percent encoding and decoding. (n.d.). https://www.url-encode-decode.com/
9. Aegis. (2021, October 28). *SQL Injection Analysis tutorial*. https://www.linkedin.com/pulse/sql-injection-analysis-tutorial-aegis-9
10. *FTP Server – Beware of Security Risks*. (2025, March 19). https://www.ssh.com/academy/ssh/ftp/server
11. Rmcmurray. (2022, April 6). *IIS 8.0 FTP Logon attempt Restrictions*. Microsoft Learn. https://learn.microsoft.com/en-us/iis/get-started/whats-new-in-iis-8/iis-80-ftp-logon-attempt-restrictions
12. V, J. (2024, December 2). *How to protect FTP passwords from brute force attacks | JSCAPE*. JSCAPE. https://www.jscape.com/blog/protecting-ftp-passwords-from-brute-force-attacks